Decentrathon 6.0 · трек 01 · команда null-pointers

Тот же `soup train` — почти вдвое быстрее, в 36 раз легче

Qwen2.5-1.5B-Instruct в NF4 · LoRA r=16 на q/v · batch 4 · до 512 токенов · 64 шага · Kaggle T4
Наш пакет **`solution/`** подключается к Soup одной функцией **`apply()`**

×1,84
скорость всей команды
83,2 → 45,3 с

×36
меньше пик памяти GPU
14 345 → 395 MiB

+0,008 %
eval loss против базы
допуск 0,5 %

Куда уходят время и память базы

Soup 0.77.0 на T4

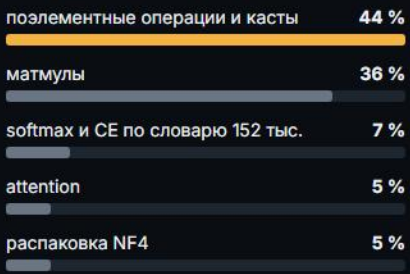
43 % позиций батча — паддинг; он идёт через все 28 слоёв, голову и loss

~4 ГБ логиты `(B, T, 151 936)`, fp32-копия для loss и три чтения метрик

~6 ГБ gate, up и SiLU·up в fp32 ждут backward: 3 × 73 МБ на слой × 28

3 запуска bitsandbytes на распаковку каждого NF4-веса, в forward и backward

Профиль шага базы, время GPU



Пик памяти базы

14 345 из 15 360 MiB T4

Архитектура: на GPU — только текущий слой

батч базы: 43 % паддинга



→

одна последовательность из N реальных токенов, у каждой строки свои позиции RoPE



varlen attention: один вызов на все строки · хвост до кратного 16 для cuBLAS

медиана шага 0,416 → 0,329 с

GPU

T4, 15 ГБ

кольцо NF4-весов по типам	**25,7 MiB вместо 645**
кольцо чанков головы	**2 × 12 MiB**

токены и активации текущего слоя

пик 395 MiB = 274 аллокатор torch + 121 контекст CUDA

PCIe, боковые потоки

←

веса слоя i±1 и чанк головы c+2: префетч под счётом

→

оффлоад gate/up в int8 и входов слоёв

Хост

закреплённая память, ~3,6 ГБ

NF4-веса 28 слоёв	**645 MiB**
голова fp16 151 936 × 1 536	**445 MiB**
слабы оффлоада активаций	**2 560 MiB**

attention в backward пересчитывается бит в бит

Свои Triton-ядра

18 ядер · 22,2 % времени шагов 3+ (порог 5 %)

Голова без логитов

`head_stream.py`

Словарь чанками по 4 096 строк из pinned-памяти. Ядро ведёт онлайн max, Σexp, энтропию, argmax и сразу пишет P для градиента; loss, точность и dL/dh — без тензора логитов.

(N, 151 936) не создаётся: ~4 ГБ → 2 × 12 MiB

Распаковка NF4

`nf4.py`

Коды, absmax и масштабы второго уровня → fp16 за один проход вместо трёх запусков bitsandbytes с двумя выделениями. Побитово равно `dequantize_4bit`.

211 ГБ/с ≈ 88 % полосы памяти T4

int8 в backward MLP

`int8_bwd.py · int8_glue.py`

NF4 сразу в транспонированный int8, GEMM градиента по входу — `torch._int_mm` на тензорных ядрах. Эпилог SwiGLU, квантование и деквантование градиента — наши ядра.

GEMM ×1,6–1,9 к fp16 · 26 операций torch → 2 ядра

int8-оффлоад gate/up

`offload_int8.py`

SwiGLU forward и int8-форма (gate, up) за один проход, масштаб на токен и группу из 128; backward читает её напрямую. Плюс RMSNorm и RoPE: по одному ядру в каждую сторону.

51,6 % байт half по шине · −1,64 с в ABBA

Процессор и старт

Харнесс меряет всю команду, от запуска до адаптера

−1,2…1,4 с

граница шага без синхронизаций хоста: фильтр nan на GPU, следующий батч и план unpad — в тени backward

−670

запусков на шаг: клей int8 backward (~26 операций torch на слой) свёрнут в 2 своих ядра

7,2 → 4,9 с

прогрев ядер в фоновом потоке, пока Soup грузит модель; лаунчеры Triton с -O0 одним вызовом компилятора

Против базы

Kaggle T4, база и решение в одной сессии · soup-m13-full

Время всей команды soup train **s = 1,84**

База	83,2 с
Решение	45,3 с

из них 64 шага обучения **×2,31**

База	59,9 с
Решение	25,9 с

Пик памяти GPU по nvidia-smi **m = 36,3**

База	14 345 MiB
Решение	395 MiB

Качество

eval loss	2,04963 → 2,04980
против базы	+0,008 %
кривая loss, шаг 1 / среднее	0,01 % / 0,06 %
допуски	0,5 % · 1 % / 3 %

Протокол жюри: 18 наборов скрытых строк

s, порядок Б-Р-Р-Б	1,74–1,91
пик, MiB	391–453
eval против базы	−0,012 … +0,036 %
проверки	10 из 10 каждый раз

Что не сработало и ограничения

Измерили на T4 и отказались

Свой backward attention
0,70–1,16 мс на слой против 0,45–0,58 у fmha PyTorch

int8 в forward MLP
−0,24…−0,32 с, но +6…24 MiB пика и компиляция трёх ядер

int8-копия головы
шаги −0,14 с, сборка копии ~0,4 с

CUDA graph на MLP
+52 MiB, ядра в графе не идут в зачёт 5 %

12-битная lossless-голова
шина 445 → 334 MiB, стена в шуме, +2 MiB

Ограничения

- Короткие шаги упираются в CPU (постановка ядер), длинные — в мощность T4 (70 Вт)
- ~3,6 ГБ закреплённой памяти хоста и PCIe x16; на x8 копии становятся узким местом
- int8-пути приближённые: кривая loss 0,05–0,07 %, accuracy до 0,36 % при допуске 3 %
- Заточено под задачу: Qwen2, NF4, LoRA на q/v, fp16 на T4; любой другой вход идёт по исходному коду

Дальше и как воспроизвести

Следующие шаги

- Меньше Python на запуск: слить оставшийся клей шага в ядра
- Свой IMMA-GEMM gate/up со слитым эпилогом
- Тензорные ядра Turing из Triton: inline PTX `mma.sync` с распаковкой NF4 в тайле

Воспроизвести: Run All, ~12 минут

`kaggle.com/code/slerfas/soup-final-public`
`github.com/BAITC-Hacks/hack-78c6e9c3-null-pointers`

README: каждая оптимизация, её переключатель `SOUP_SOLUTION_*`, замер и разрешение организаторов

Спасибо! команда null-pointers